Training Course

Amazon Web Service

Module 10:
Elastic Container Service

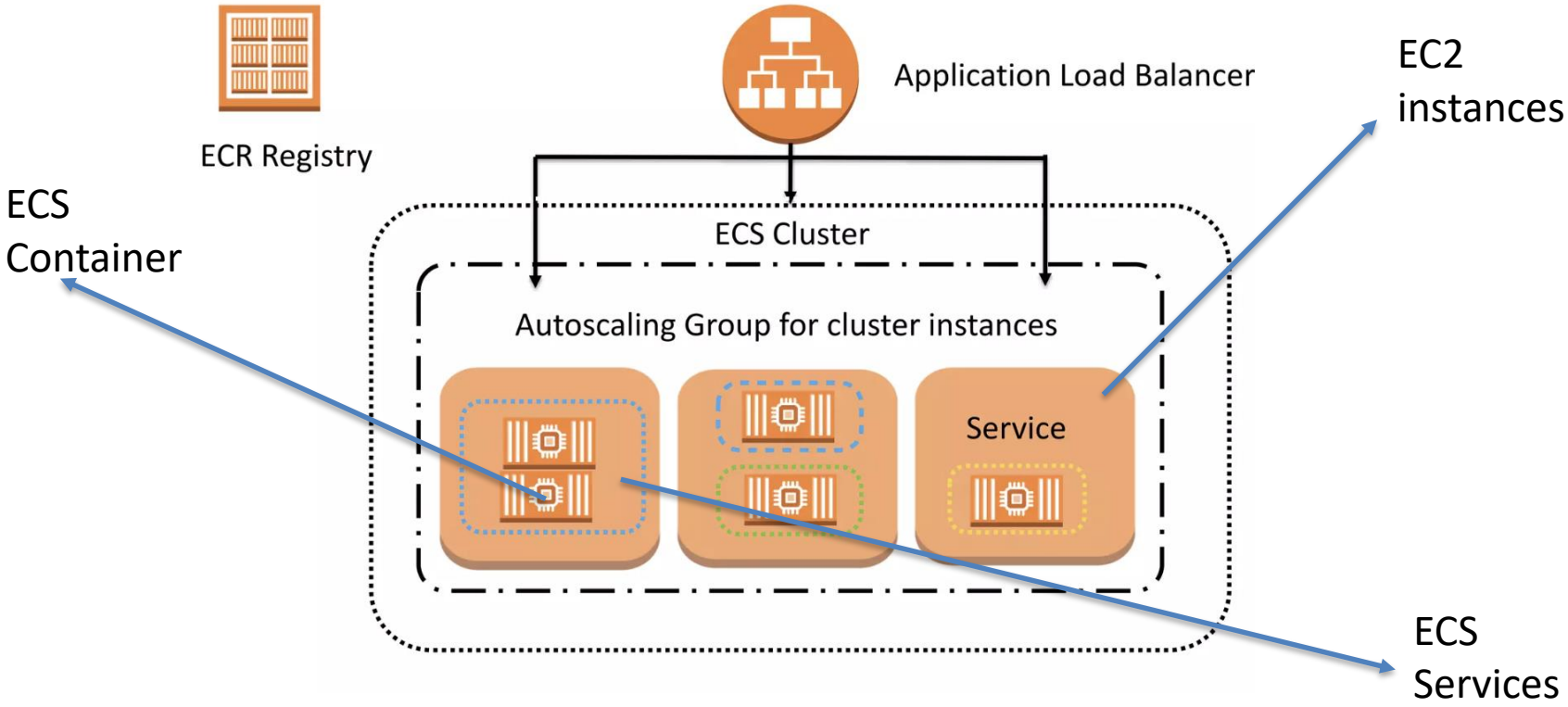
Module 10: ECS

- **Goal**: Understanding Containerization Service in AWS
  - ✓ What is ECS
  - ✓ ECS components
  - ✓ ECS type
  - ✓ ECS networking
  - ✓ ECS security (IAM task execution role)
  - ✓ ECS monitoring
  - ✓ ECR (Elastic Container Registry)
- **Prerequisites:**
  - ✓ Containerization and docker
  - ✓ Networking in AWS

  Lab: Create and configure ECS and ECR

What is ECS?

- Highly scalable, high performance container management system.
- Eliminates the need to install, operate and scale your own container management system


ECR Registry
Application Load Balancer
EC2 instances
ECS Container
ECS Cluster
Autoscaling Group for cluster instances
Service
ECS Services

Types of ECS?

- EC2:
  - Users/Administrators have to manage, provision, scaling, monitoring underlying EC2s
  - More control over the underlying infrastructure
- Fargate:
  - Serverless
  - Users/Administrators don't have to manage the underlying infrastructure, only need to define the workload for container services.
  - More expensive compares to using EC2 type

Components in ECS?

- ECS Cluster:
  - Is made up of one or more EC2 instances
  - Each **cluster** instance runs one or more services
- ECS Service:
  - A **Service** controls things like the number of copies of a T**ask** you want running
  - Register **service** with a load balancing
- ECS Task Definition:
  - Controls things like which container image will be run, environment variable, resource allocation, and logging, ...

ECS Task Definition

- Task definition is required to run Docker in ECS.
- This is can be understood as a blueprint on how ECS runs our container.

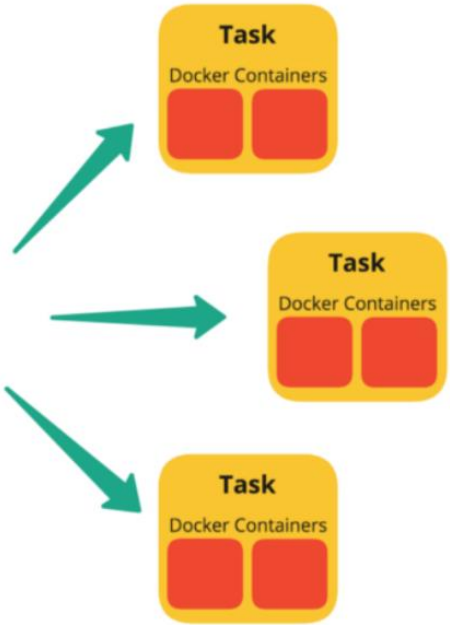
ECS Task Definition

What included int the task definition?

- The Docker image to use with each container in your task
- How much CPU and memory to use with each task or each container within a task
- The launch type to use, which determines the infrastructure that your tasks are hosted on
- The Docker networking mode to use for the containers in your task
- The logging configuration to use for your tasks
- Whether the task continues to run if the container finishes or fails
- The command that the container runs when it's started
- Any data volumes that are used with the containers in the task
- The IAM role that your tasks use
- **You can define multiple images in the task definition**

ECS Task

```
taskDefinition:
  containerDefinitions:
    - name: "my-app"
      image: "xxx.ecr.{{region}}.amazonaws.com/xxx:{{tag}}"
      cpu: 300
      memory: 750
      portMappings:
        - containerPort: 5000
          protocol: "tcp"
      links:
        - statsd:statsd
      environment:
        - name: 'NODE_ENV'
          value: '{{ environment }}'
        - name: 'AWS_REGION'
          value: '{{ region }}'
        - name: 'UV_THREADPOOL_SIZE'
          value: '128'
    - name: "statsd"
      image: "xxx.ecr.{{region}}.amazonaws.com/xxx-statsd:4.5.0"
      cpu: 64
      memory: 64
      environment:
        - name: "TARGET"
          value: '{{ statsd_target }}'
  startupTimeout: 20
  taskRoleArn: {{ task_role_arn }}
```



An instance of a Task Definition, running the containers detailed within it. Multiple Tasks can be created by one Task Definition, as demand requires.

ECS Service

- This is the logical level for application service
- Defines the minimum and maximum **Tasks** from one **Task Definition**, autoscaling, and load balancing.
  - Autoscaling: define minimum, maximum **Tasks** (**container group** defined in the **Task Definition**), and scaling policy.

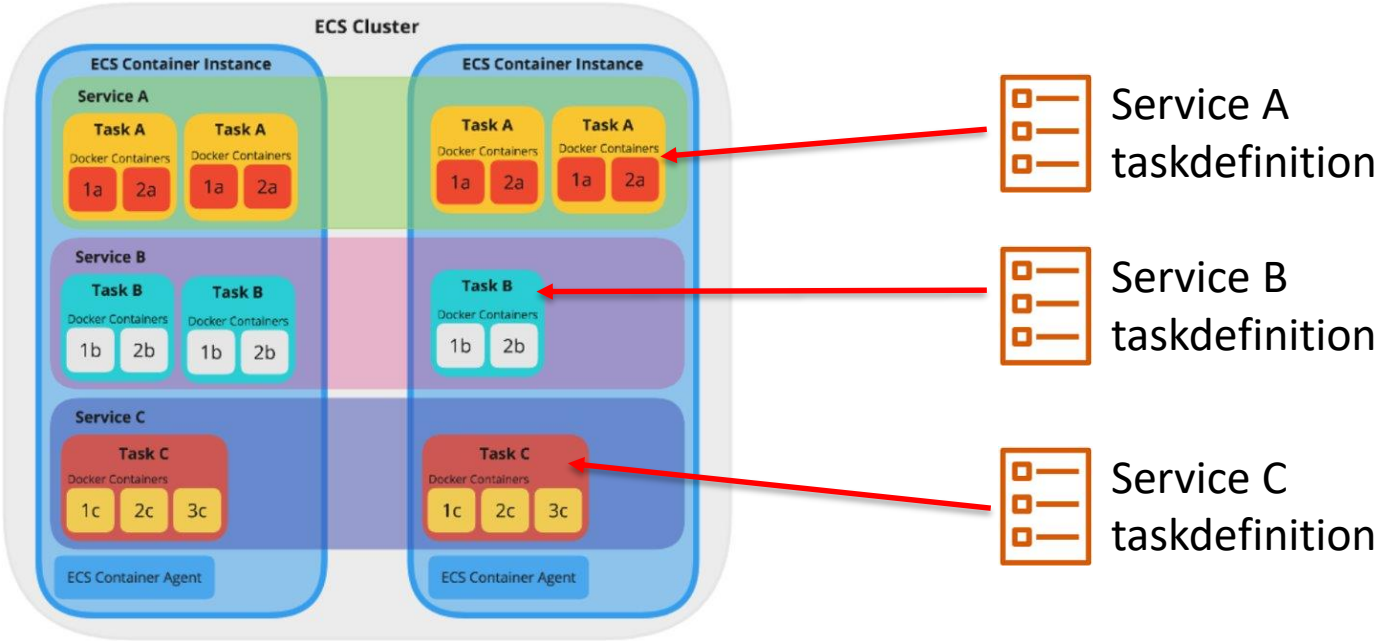
ECS Service

- Now that we have our **Service**, its **Tasks** need to run somewhere in order to be accessible. It needs to be put on a **Cluster,** and the container management service will handle it running across one or more **ECS Container Instance(s)**.
  - It needs EC2 instance that has **Docker** and **ECS container Agent** running on it.
  - The Agent takes care of the communication between ECS and the instance, providing the status of running containers and managing running new ones.

ECS Cluster

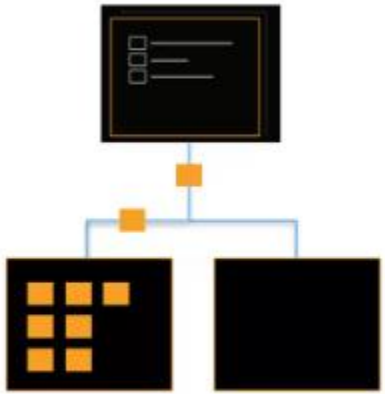
- Cluster is a group of ECS Container Instances
- A Cluster can run many Services

ECS Architecture



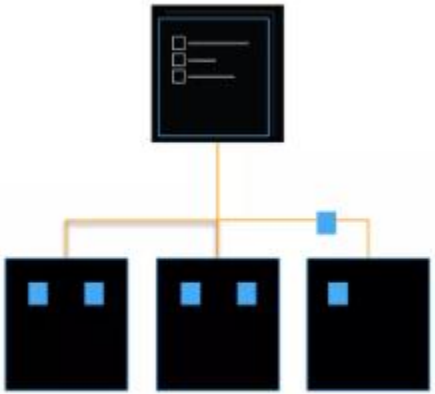
ECS Task Placement (Only available on ECS EC2 type)

- Task placement strategy: algorithm for selecting instances for task placement, or tasks for termination.
  - AZ balanced spread: Evenly spread task across AZ
  - AZ balanced binpack:
  - Binpack:
    - Tasks are placed on container instances so as to leave the least amount of unused CPU or memory.
    - This strategy minimizes the number of container instances in use.
  - One task per host: each host only contain 1 task
  - Random:
- Task placement constraint: rule taken into consideration during task placement



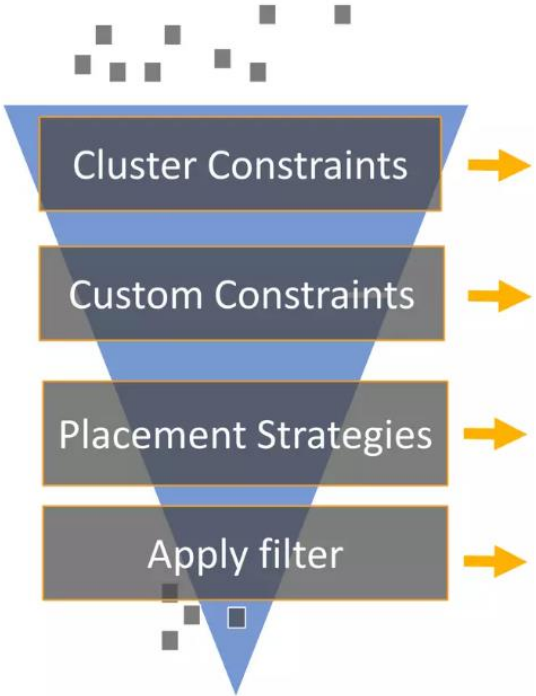
Binpacking

Random



Spread

How does task placement work?



- Satisfy CPU, memory, and port requirements
- Filter for location, instance-type, AMI, or custom attribute constraints
- Identify instances that meet spread or binpack placement strategy
- Select final container instances for placement

ECS Task Networking

- ECS EC2 type:
  - awsvpc:
    - The task is allocated its own elastic network interface (ENI) and a primary private IPv4 address.
    - This gives the task the same networking properties as Amazon EC2 instances.
  - bridge:
    - The task uses Docker's built-in virtual network on Linux, which runs inside each Amazon EC2 instance that hosts the task.
    - The built-in virtual network on Linux uses the bridge Docker network driver.
    - This is the default network mode on Linux if a network mode isn't specified in the task definition.

ECS Task Networking

- host:
  - The task uses the host's network which bypasses Docker's built-in virtual network by mapping container ports directly to the ENI of the Amazon EC2 instance that hosts the task.
  - Dynamic port mappings can’t be used in this network mode.
  - A container in a task definition that uses this mode must specify a specific hostPort number.
  - A port number on a host can’t be used by multiple tasks. As a result, you can’t run multiple tasks of the same task definition on a single Amazon EC2 instance.
- none:
  - The task has no external network connectivity.

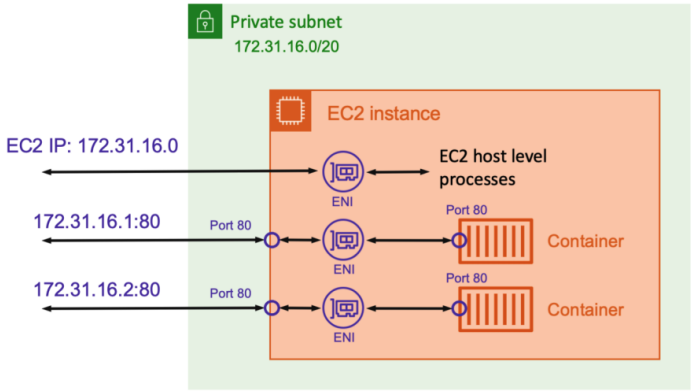
ECS Task Networking

- Default (Windows containers):
  - The task uses Docker's built-in virtual network on Windows, which runs inside each Amazon EC2 instance that hosts the task.
  - The built-in virtual network on Windows uses the nat Docker network driver.
  - This is the default network mode on Windows if a network mode isn't specified in the task definition.

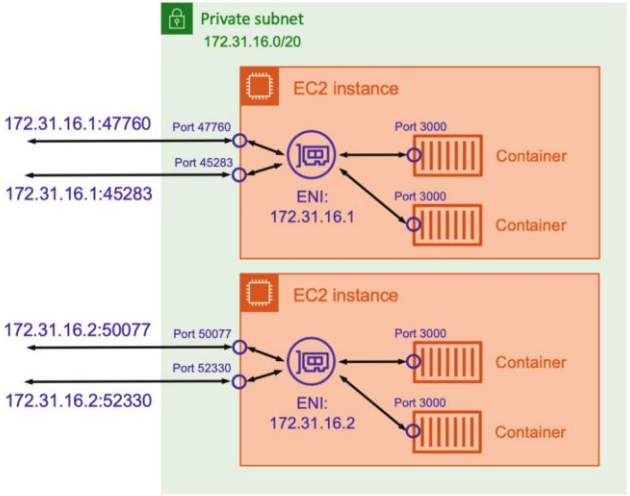
ECS Task Networking

- ECS Fargate type:
  - ✓ only support awsvpc type.

ECS Task Networking

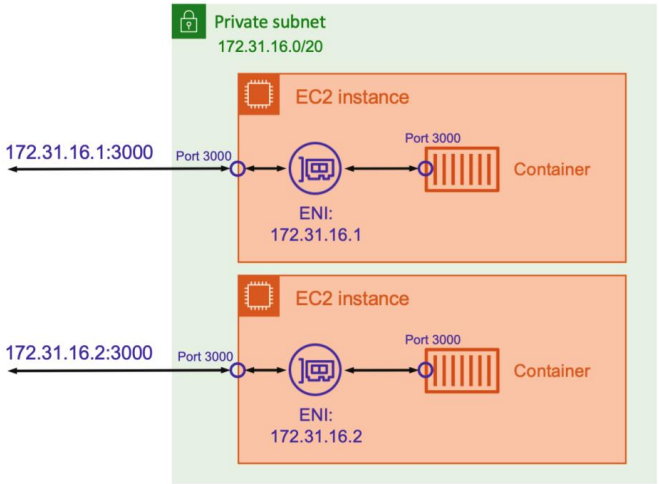


Awsvpc



Bridge

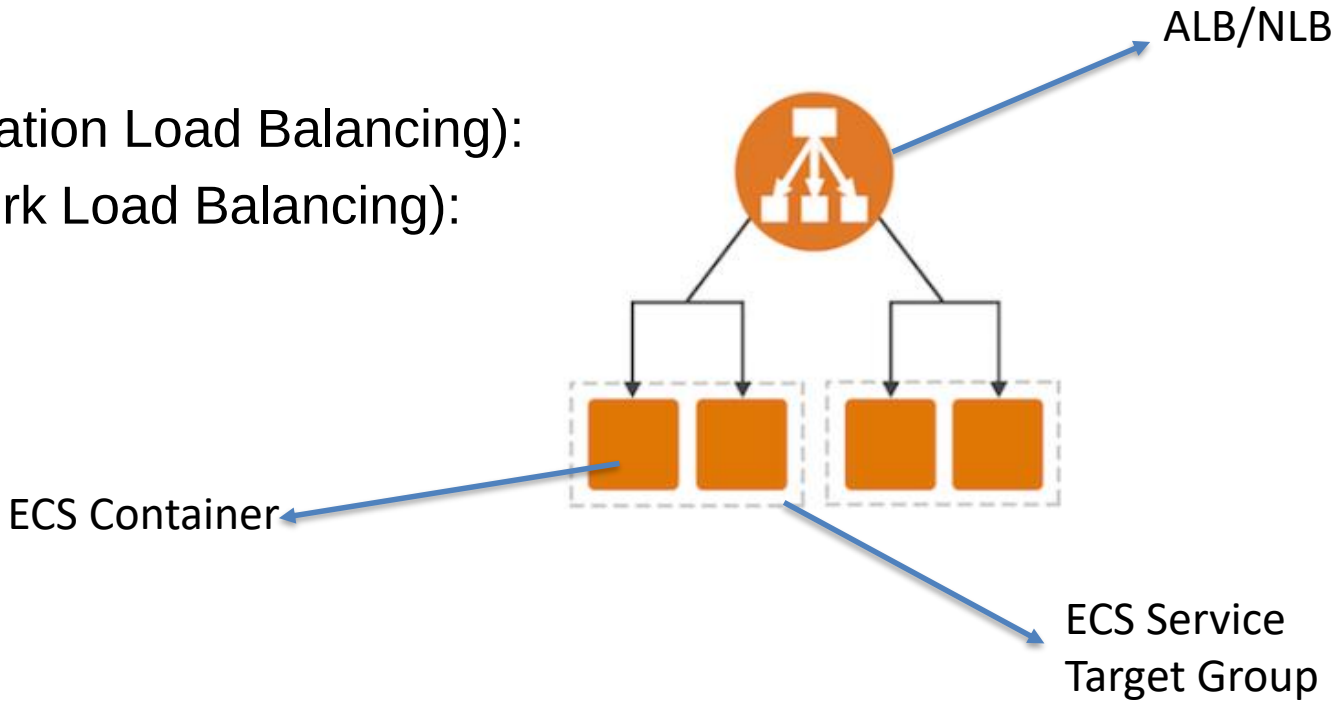
ECS Task Networking



Host mode

ECS Service ELB

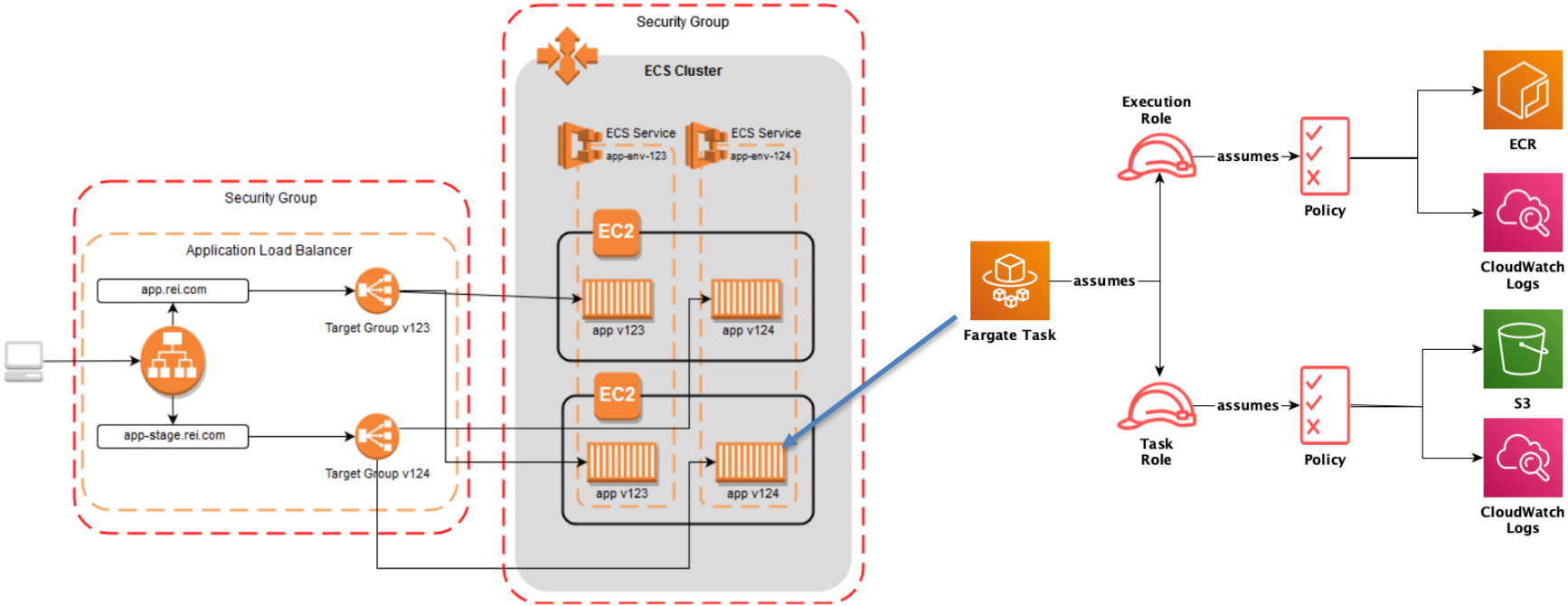
- Support:
  - ALB (Application Load Balancing):
  - NLB (Network Load Balancing):



ECS Security And Permission

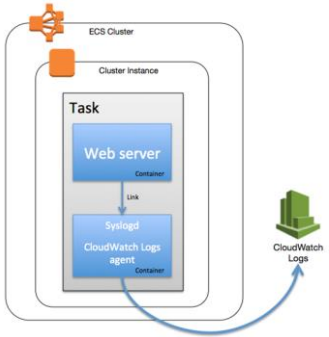
- IAM role:
  - Task IAM role:
    - allows containers in the task to make API requests to AWS services.
  - Task execution IAM role:
    - used by the container agent to make AWS API requests on your behalf
- Security Group (awsvpc network mode):
  - ECS Service has to be attached to security groups.
  - This is used for allowing only allowed connection to the containers/ECS tasks.

ECS Security And Permission



ECS Monitoring

- You can monitor your Amazon ECS resources using CloudWatch
  - Amazon ECS metric data is automatically sent to CloudWatch in 1-minute periods
  - CloudWatch Container Insight: enable on **cluster** level
- ECS can logs amazon ECS API calls with CloudTrail
- You can use a sidecar agent container for collecting logs, metrics, and telemetry data. (example: AWS Distro for OpenTelemetry)

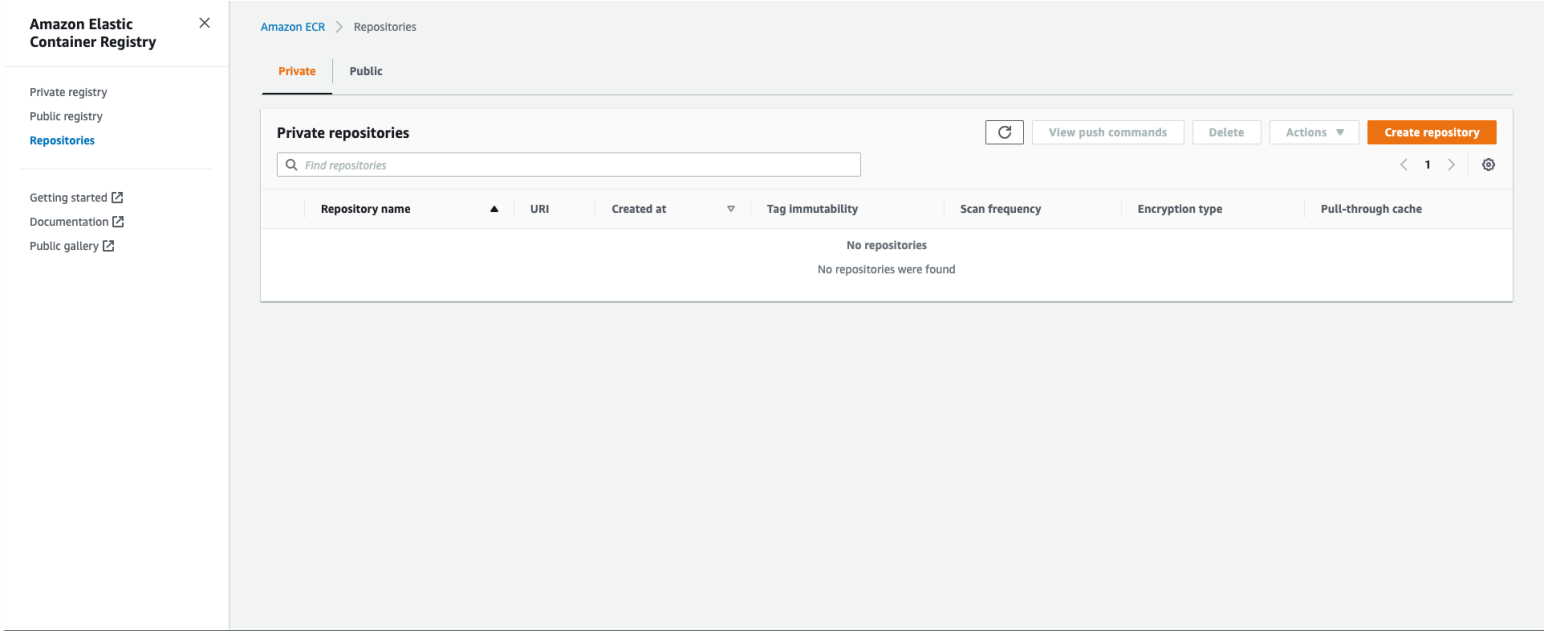


ECR (Elastic Container Registry)

- AWS managed container image registry service that is secure, scalable, and reliable.
- Supports private repositories with resource-based permissions using AWS IAM. This is so that specified users or Amazon EC2 instances can access your container repositories and images.
- You can use your preferred CLI to push, pull, and manage Docker images, Open Container Initiative (OCI) images, and OCI compatible artifacts.

ECR (Elastic Container Registry)

- Has registry policy to grant and deny access to the container registry



Thank you!!!